KVTensor: Technical Report

ABSTRACT

Modern neural inference is increasingly bottlenecked by memory capacity rather than arithmetic throughput. When model weights exceed memory, inference becomes a data management problem as much as a computational problem, because performance depends on how external tensor data is physically organized, accessed, prefetched, and buffered. Yet most out-of-core systems remain reactive, forcing storage to serve tensor requests on demand. For repeated-step workloads such as transformer-based large language models (LLMs), however, much of the weight access order is determined offline. This makes it possible to replace demand-driven tensor access with a planned weight-consumption stream served by a key-value database. We present KVTensor, a weight-streaming inference runtime that translates a model into a chunk-streaming execution plan, stores streamable weights as chunked key-value objects, and executes inference through a bounded prefetch-and-buffer pipeline using standard numerical kernels. We formalize a semantics-preserving translation with numerical equivalence guarantees, and develop a configuration-ranking model that jointly optimizes chunk size, static tensor residency, and ring-buffer allocation under a fixed memory budget. Evaluated on CPU-only edge-class hardware with Llama 3.1 8B, KVTensor achieves up to a 7.05× prefill and 5.56× decode speedup over competing out-of-core baselines under memory-constrained conditions, degrades gracefully as memory budgets tighten, and generalizes to computer vision workloads.

CCS CONCEPTS

• **Information systems** → **Information storage technologies**; **Data access methods**.

KEYWORDS

Key-value Database, Large Language Model, Out-of-core Execution

1 SOURCE LANGUAGE: WELL TYPEDNESS

Shapes and tensors. A *shape* is a finite tuple of positive integers

$$\delta = (d_1, \ldots, d_k), \qquad k \geq 0.$$

A *tensor* of shape δ over the chosen numeric domain $\mathbb{D}$ is a k-dimensional array indexed by $[d_1] \times \cdots \times [d_k]$ with entries in $\mathbb{D}$. We write Shape for the set of all shapes and $\mathcal{T}_\delta$ for the set of all tensors of shape δ. The empty tuple () denotes a scalar.

Identifiers. We use two classes of identifiers. Let StateId denote the finite set of *state identifiers*. Each $q \in \mathsf{StateId}$ names one persistent runtime state tensor carried from one execution of the step body to the next. Let ParamId denote the finite set of *parameter identifiers*. Each $T \in \mathsf{ParamId}$ names one read-only model tensor.

Source syntax. The source language for one repeated step is

$$\begin{aligned} e ::= {} & X \mid \mathsf{State}(q) \mid \alpha \mid e_1 + e_2 \mid e_1 \odot e_2 \mid \phi(e) \mid \mathsf{reshape}(e, \delta) \\ & \mid \mathsf{transpose}(e, \pi) \mid \mathsf{linear}(T, e) \mid \mathsf{norm}(e). \end{aligned}$$

Here, α denotes a scalar constant in $\mathbb{D}$, ϕ an elementwise function, $\delta \in \mathsf{Shape}$ a target shape, and π a permutation of operand dimensions. Parameter tensors appear only through the linear operator $\mathsf{linear}(T, e)$. A complete *step body* is represented as

$$F = \left(e^{out}, \{e_q^{state}\}_{q \in \mathsf{StateId}}\right),$$

where e^{out} computes the emitted output of the current step and, for each $q \in \mathsf{StateId}$, e_q^{state} computes the next value of state slot q.

A *shape environment* is a triple

$$\Gamma = (\delta_X, \Sigma, \Pi),$$

where $\delta_X \in \mathsf{Shape}$ is the shape of the current-step input, $\Sigma : \mathsf{StateId} \to \mathsf{Shape}$ assigns a shape to each state tensor, and $\Pi : \mathsf{ParamId} \to \mathsf{Shape}$ assigns a shape to each parameter tensor.

The typing judgment

$$\Gamma \vdash e : \delta$$

means that expression e has shape δ under environment Γ. The base typing rules are

$$\frac{}{\Gamma \vdash X : \delta_X} \qquad \frac{}{\Gamma \vdash \mathsf{State}(q) : \Sigma(q)} \qquad \frac{}{\Gamma \vdash \alpha : ()}.$$

The remaining operators follow the standard dense-tensor discipline. Addition and Hadamard product require equal-shaped operands and preserve shape. Elementwise functions and normalization preserve shape. Reshape preserves the total number of elements. Transpose permutes dimensions. Formally,

$$\Gamma \vdash e_1 : \delta,\ \Gamma \vdash e_2 : \delta \Longrightarrow \Gamma \vdash e_1 + e_2 : \delta,\ \Gamma \vdash e_1 \odot e_2 : \delta,$$

$$\Gamma \vdash e : \delta \Longrightarrow \Gamma \vdash \phi(e) : \delta,\ \Gamma \vdash \mathsf{norm}(e) : \delta,$$

$$\Gamma \vdash e : \delta,\ |\delta| = |\delta'| \Longrightarrow \Gamma \vdash \mathsf{reshape}(e, \delta') : \delta',$$

$$\Gamma \vdash e : (d_1, \ldots, d_k),\ \pi \in S_k \Longrightarrow \Gamma \vdash \mathsf{transpose}(e, \pi) : (d_{\pi(1)}, \ldots, d_{\pi(k)}).$$

where $|\delta|$ denotes the total number of elements of shape δ.

The linear operator is the only case that changes the last dimension according to its parameter tensor. If $\Pi(T) = (m, n)$ and

$$\Gamma \vdash e : (\delta_1, \ldots, \delta_k, n),$$

then

$$\Gamma \vdash \mathsf{linear}(T, e) : (\delta_1, \ldots, \delta_k, m).$$

A step body

$$F = \left(e^{out}, \{e_q^{state}\}_{q \in \mathsf{StateId}}\right)$$

is well typed under Γ if $\Gamma \vdash e^{out} : \delta^{out}$ for some output shape δ^{out} and

$$\Gamma \vdash e_q^{state} : \Sigma(q) \qquad \text{for every } q \in \mathsf{StateId}.$$

2 SUPPLEMENTARY FORMALIZATION

This appendix gives a complete formalization of the translation from the tensor-level repeated-step language to chunk-streaming execution plans. The presentation has four layers.

(1) A typed *source language* for one repeated step body.
(2) A *dense tensor semantics* over full parameter tensors.
(3) A deployment-dependent *chunk realization* together with a logically equivalent *chunk semantics*.

(4) A *target language* of chunk-streaming plans, a compilation judgment from source to target, and a target runtime semantics that consumes a sequential key stream.

The final theorem states that compiling a well-typed step body preserves its source semantics exactly.

2.1 Basic Objects

Shapes. A *shape* is a finite tuple of positive integers

$$\delta = (d_1, \ldots, d_k), \qquad k \geq 0.$$

We write Shape for the set of all shapes. The empty tuple () denotes a scalar. For $\delta = (d_1, \ldots, d_k)$, let

$$|\delta| = \prod_{i=1}^{k} d_i, \qquad |()| = 1.$$

For each $\delta \in \mathsf{Shape}$, let $\mathcal{T}_\delta$ denote the set of all tensors of shape δ over the chosen numeric domain.

If $\bar{\delta} = (d_1, \ldots, d_k)$ is a prefix shape and n is a positive integer, we write $(\bar{\delta}, n)$ for the shape $(d_1, \ldots, d_k, n)$. The prefix $\bar{\delta}$ may be empty.

Identifiers. Let StateId be the finite set of state identifiers and ParamId the finite set of parameter identifiers. Each $q \in \mathsf{StateId}$ names one persistent state tensor. Each $T \in \mathsf{ParamId}$ names one read-only model tensor.

Step bodies. A repeated step body has the form

$$F = \big(e^{out}, \{e_q^{state}\}_{q \in \mathsf{StateId}}\big),$$

where e^{out} computes the emitted output of the current step and e_q^{state} computes the next value of state slot q.

We fix once and for all a total order

$$q_1 \prec q_2 \prec \cdots \prec q_m$$

over StateId. This order is used only to linearize state-update evaluation in the target plan.

2.2 Source Language

Syntax. The source language of tensor expressions is

$$\begin{aligned} e ::= {} & X \mid \mathsf{State}(q) \mid \alpha \mid e_1 + e_2 \mid e_1 \odot e_2 \mid \phi(e) \\ & \mid \mathsf{reshape}(e, \delta) \mid \mathsf{transpose}(e, \pi) \mid \mathsf{linear}(T, e) \mid \mathsf{norm}(e). \end{aligned} \tag{S1}$$

Here

- α is a scalar constant
- ϕ is an elementwise function
- $\delta \in \mathsf{Shape}$ is a target shape
- π is a permutation of tensor dimensions
- parameters appear only through $\mathsf{linear}(T, e)$.

Typing environments. A *shape environment* is a triple

$$\Gamma = (\delta_X, \Sigma, \Pi),$$

where

$$\delta_X \in \mathsf{Shape}, \qquad \Sigma : \mathsf{StateId} \to \mathsf{Shape}, \qquad \Pi : \mathsf{ParamId} \to \mathsf{Shape}.$$

Typing judgment. The judgment

$$\Gamma \vdash e : \delta$$

means that expression e has shape δ under Γ.

Typing rules. The typing rules are standard dense-tensor rules:

$$\frac{}{\Gamma \vdash X : \delta_X} \qquad \frac{}{\Gamma \vdash \mathsf{State}(q) : \Sigma(q)} \qquad \frac{}{\Gamma \vdash \alpha : ()} \tag{T-Base}$$

$$\frac{\Gamma \vdash e_1 : \delta \qquad \Gamma \vdash e_2 : \delta}{\Gamma \vdash e_1 + e_2 : \delta} \qquad \frac{\Gamma \vdash e_1 : \delta \qquad \Gamma \vdash e_2 : \delta}{\Gamma \vdash e_1 \odot e_2 : \delta} \tag{T-Bin}$$

$$\frac{\Gamma \vdash e : \delta}{\Gamma \vdash \phi(e) : \delta} \qquad \frac{\Gamma \vdash e : \delta}{\Gamma \vdash \mathsf{norm}(e) : \delta} \tag{T-Unary}$$

$$\frac{\Gamma \vdash e : \delta \qquad |\delta| = |\delta'|}{\Gamma \vdash \mathsf{reshape}(e, \delta') : \delta'} \tag{T-Reshape}$$

$$\frac{\Gamma \vdash e : (d_1, \ldots, d_k) \qquad \pi \in S_k}{\Gamma \vdash \mathsf{transpose}(e, \pi) : (d_{\pi(1)}, \ldots, d_{\pi(k)})} \tag{T-Transpose}$$

$$\frac{\Pi(T) = (m, n) \qquad \Gamma \vdash e : (\bar{\delta}, n)}{\Gamma \vdash \mathsf{linear}(T, e) : (\bar{\delta}, m)} \tag{T-Linear}$$

A step body

$$F = \big(e^{out}, \{e_q^{state}\}_{q \in \mathsf{StateId}}\big)$$

is *well typed* under Γ if

$$\Gamma \vdash e^{out} : \delta^{out}$$

for some output shape δ^{out} and

$$\Gamma \vdash e_q^{state} : \Sigma(q) \qquad \text{for all } q \in \mathsf{StateId}.$$

2.3 Dense Source Semantics

Fix a well-typed shape environment

$$\Gamma = (\delta_X, \Sigma, \Pi).$$

The dense source semantics is defined relative to

- a current-step input tensor $x \in \mathcal{T}_{\delta_X}$
- a state map s such that $s(q) \in \mathcal{T}_{\Sigma(q)}$ for all $q \in \mathsf{StateId}$
- a parameter map Θ such that $\Theta(T) \in \mathcal{T}_{\Pi(T)}$ for all $T \in \mathsf{ParamId}$.

For each well-typed expression $\Gamma \vdash e : \delta$, its dense meaning

$$[\![e]\!]_{x,s,\Theta} \in \mathcal{T}_\delta$$

is defined compositionally by

$$[\![X]\!]_{x,s,\Theta} = x, \qquad [\![\mathsf{State}(q)]\!]_{x,s,\Theta} = s(q), \qquad [\![\alpha]\!]_{x,s,\Theta} = \alpha,$$

$$[\![e_1 + e_2]\!]_{x,s,\Theta} = [\![e_1]\!]_{x,s,\Theta} + [\![e_2]\!]_{x,s,\Theta},$$

$$[\![e_1 \odot e_2]\!]_{x,s,\Theta} = [\![e_1]\!]_{x,s,\Theta} \odot [\![e_2]\!]_{x,s,\Theta},$$

$$[\![\phi(e)]\!]_{x,s,\Theta} = \phi\big([\![e]\!]_{x,s,\Theta}\big),$$

$$[\![\mathsf{reshape}(e, \delta)]\!]_{x,s,\Theta} = \mathsf{reshape}\big([\![e]\!]_{x,s,\Theta}, \delta\big),$$

$$[\![\mathsf{transpose}(e, \pi)]\!]_{x,s,\Theta} = \mathsf{transpose}\big([\![e]\!]_{x,s,\Theta}, \pi\big),$$

$$[\![\mathsf{norm}(e)]\!]_{x,s,\Theta} = \mathsf{norm}\big([\![e]\!]_{x,s,\Theta}\big).$$

The linear operator is interpreted over the full dense parameter tensor:

$$[\![\mathsf{linear}(T, e)]\!]_{x,s,\Theta} = \mathsf{Linear}\big(\Theta(T),\ [\![e]\!]_{x,s,\Theta}\big). \tag{S-Lin}$$

Here Linear denotes the ordinary in-memory dense linear kernel that multiplies along the last dimension.

The dense semantics of a complete step body is

$$\mathsf{Eval}_{\mathrm{src}}(F, \Theta, x, s) = (y, s'),$$

where

$$y = [\![e^{out}]\!]_{x,s,\Theta}, \qquad s'(q) = [\![e_q^{state}]\!]_{x,s,\Theta} \quad \text{for all } q \in \mathsf{StateId}. \tag{S-Step}$$

2.4 Deployment, Chunk Realization, and Stores

Chunk-streaming execution depends on a deployment choice. We fix

$$\mathcal{T}_{\text{res}} \subseteq \mathsf{ParamId}, \qquad \mathcal{T}_{\text{str}} = \mathsf{ParamId} \setminus \mathcal{T}_{\text{res}},$$

and a chunk-width map

$$\chi : \mathsf{ParamId} \to \mathbb{N}_{>0}.$$

For a parameter tensor T with shape

$$\Pi(T) = (m, n),$$

we interpret $\chi(T)$ as the maximum number of columns in each chunk. For resident tensors we take the special case

$$T \in \mathcal{T}_{\text{res}} \Longrightarrow \chi(T) = n,$$

so a resident tensor is represented as one full-tensor chunk.

Chunk counts and chunk widths. For each $T \in \mathsf{ParamId}$ with $\Pi(T) = (m, n)$, define

$$r_T = \left\lceil \frac{n}{\chi(T)} \right\rceil.$$

Let

$$\omega_{T,1}, \ldots, \omega_{T,r_T}$$

be the widths of the chunks of T, where

$$\omega_{T,i} = \chi(T) \quad \text{for } 1 \le i < r_T, \qquad \omega_{T,r_T} = n - (r_T - 1)\chi(T).$$

If $r_T = 1$, then $\omega_{T,1} = n$.

Chunk realization. Given a dense parameter map Θ, a *chunk realization* of Θ under χ is a map M such that, for each $T \in \mathsf{ParamId}$,

$$M(T) = \langle M(T)_1, \ldots, M(T)_{r_T} \rangle$$

with

$$M(T)_i \in \mathcal{T}_{(m,\omega_{T,i})} \qquad \text{and} \qquad \mathsf{Concat}\big(M(T)_1, \ldots, M(T)_{r_T}\big) = \Theta(T),$$

where Concat denotes column-wise concatenation.

Input splitting. If $v \in \mathcal{T}_{(\bar{\delta},n)}$ and T has chunk widths $\omega_{T,1}, \ldots, \omega_{T,r_T}$, define

$$\mathsf{Split}_{\chi,T}(v) = \langle v_1, \ldots, v_{r_T} \rangle$$

to be the partition of the last dimension of v into contiguous slices with

$$v_i \in \mathcal{T}_{(\bar{\delta},\omega_{T,i})}.$$

This split is uniquely determined by the chunk widths of T.

Physical placement. We now define the two physical stores induced by the deployment.

For resident tensors, memory contains the full tensor:

$$R(T) = M(T)_1 \qquad \text{for } T \in \mathcal{T}_{\text{res}}.$$

For streamed tensors, the key-value database stores the chunks:

$$D[\mathsf{key}_\chi(T, i)] = M(T)_i \qquad \text{for } T \in \mathcal{T}_{\text{str}},\ 1 \le i \le r_T.$$

The key sequence of streamed tensor T is

$$K_\chi(T) = \langle \mathsf{key}_\chi(T, 1), \ldots, \mathsf{key}_\chi(T, r_T) \rangle.$$

We say that (R, D) is *consistent* with $(\Theta, M, \chi, \mathcal{T}_{\text{res}})$ if

$$R(T) = \Theta(T) \quad \text{for all } T \in \mathcal{T}_{\text{res}},$$

and

$$D[\mathsf{key}_\chi(T, i)] = M(T)_i \quad \text{for all } T \in \mathcal{T}_{\text{str}},\ 1 \le i \le r_T.$$

2.5 Chunk Semantics

The chunk semantics keeps the source language unchanged and changes only the meaning of the linear operator. This semantics is deployment-aware through χ and M, but it is still independent of physical placement in R or D.

For each well-typed expression $\Gamma \vdash e : \delta$, define

$$[\![e]\!]^{\chi}_{x,s,M} \in \mathcal{T}_\delta$$

by the same clauses as the dense source semantics for every operator except $\mathsf{linear}(T, e)$. The non-linear clauses are therefore

$$[\![X]\!]^{\chi}_{x,s,M} = x, \qquad [\![\mathsf{State}(q)]\!]^{\chi}_{x,s,M} = s(q), \qquad [\![\alpha]\!]^{\chi}_{x,s,M} = \alpha,$$

$$[\![e_1 + e_2]\!]^{\chi}_{x,s,M} = [\![e_1]\!]^{\chi}_{x,s,M} + [\![e_2]\!]^{\chi}_{x,s,M},$$

$$[\![e_1 \odot e_2]\!]^{\chi}_{x,s,M} = [\![e_1]\!]^{\chi}_{x,s,M} \odot [\![e_2]\!]^{\chi}_{x,s,M},$$

$$[\![\phi(e)]\!]^{\chi}_{x,s,M} = \phi\big([\![e]\!]^{\chi}_{x,s,M}\big),$$

$$[\![\mathsf{reshape}(e, \delta)]\!]^{\chi}_{x,s,M} = \mathsf{reshape}\big([\![e]\!]^{\chi}_{x,s,M}, \delta\big),$$

$$[\![\mathsf{transpose}(e, \pi)]\!]^{\chi}_{x,s,M} = \mathsf{transpose}\big([\![e]\!]^{\chi}_{x,s,M}, \pi\big),$$

$$[\![\mathsf{norm}(e)]\!]^{\chi}_{x,s,M} = \mathsf{norm}\big([\![e]\!]^{\chi}_{x,s,M}\big).$$

For the linear operator, let

$$\mathsf{Split}_{\chi,T}\big([\![e]\!]^{\chi}_{x,s,M}\big) = \langle v_1, \ldots, v_{r_T} \rangle.$$

Then

$$[\![\mathsf{linear}(T, e)]\!]^{\chi}_{x,s,M} = \sum_{i=1}^{r_T} \mathsf{Linear}\big(M(T)_i,\ v_i\big). \tag{C-Lin}$$

This is uniform across resident and streamed tensors. If $T \in \mathcal{T}_{\text{res}}$, then $r_T = 1$ and the definition reduces to

$$[\![\mathsf{linear}(T, e)]\!]^{\chi}_{x,s,M} = \mathsf{Linear}\big(M(T)_1, [\![e]\!]^{\chi}_{x,s,M}\big).$$

The chunk semantics of a step body is

$$\mathsf{Eval}_\chi(F, M, x, s) = (y, s'),$$

where

$$y = [\![e^{out}]\!]^{\chi}_{x,s,M}, \qquad s'(q) = [\![e_q^{state}]\!]^{\chi}_{x,s,M} \quad \text{for all } q \in \mathsf{StateId}. \tag{C-Step}$$

Remark on alignment. No special alignment object is carried through the syntax. Alignment is induced only at a linear operator, through $\mathsf{Split}_{\chi,T}$ and the chunk widths of the parameter tensor that is about to consume the value. This is why elementwise operators, reshape, transpose, and normalization remain ordinary tensor operators in both the chunk semantics and the target plan.

2.6 Target Language

The target language specializes each source linear operator into either a resident-tensor linear operator or a streamed-tensor linear operator.

Target syntax. The target expressions are

$$\begin{aligned} g ::= {} & X \mid \mathsf{State}(q) \mid \alpha \mid g_1 + g_2 \mid g_1 \odot g_2 \mid \phi(g) \\ & \mid \mathsf{reshape}(g, \delta) \mid \mathsf{transpose}(g, \pi) \mid \mathsf{norm}(g) \mid \mathsf{linear}_{\mathrm{res}}(T, g) \mid \mathsf{linear}_{\mathrm{str}}(T, g). \end{aligned} \tag{Q1}$$

A complete target step plan has the form

$$Q = \big(g^{out}, \langle (q_1, g_1), \ldots, (q_m, g_m) \rangle\big),$$

where $q_1 \prec \cdots \prec q_m$ is the fixed total order on StateId.

A translated plan is a pair

$$P = \langle \Lambda, Q \rangle,$$

where Λ is the global sequential key stream and Q is the compute plan.

Target typing. The target typing judgment

$$\Gamma \vdash_t g : \delta$$

is identical to the source typing judgment for all non-linear operators. The two linear rules are

$$\frac{\Pi(T) = (m, n) \qquad \Gamma \vdash_t g : (\bar{\delta}, n) \qquad T \in \mathcal{T}_{\mathrm{res}}}{\Gamma \vdash_t \mathsf{linear}_{\mathrm{res}}(T, g) : (\bar{\delta}, m)} \tag{TT-Res}$$

$$\frac{\Pi(T) = (m, n) \qquad \Gamma \vdash_t g : (\bar{\delta}, n) \qquad T \in \mathcal{T}_{\mathrm{str}}}{\Gamma \vdash_t \mathsf{linear}_{\mathrm{str}}(T, g) : (\bar{\delta}, m)} \tag{TT-Str}$$

2.7 Translation

Expression compilation. The compilation judgment

$$\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e \Downarrow \langle \Lambda, g \rangle : \delta$$

means that under deployment $(\chi, \mathcal{T}_{\mathrm{res}})$, source expression e of shape δ translates to target expression g and contributes the key-stream fragment Λ.

Compilation is syntax-directed.

$$\frac{}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash X \Downarrow \langle \epsilon, X \rangle : \delta_X} \tag{C-In}$$

$$\frac{}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash \mathsf{State}(q) \Downarrow \langle \epsilon, \mathsf{State}(q) \rangle : \Sigma(q)} \tag{C-State}$$

$$\frac{}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash \alpha \Downarrow \langle \epsilon, \alpha \rangle : ()} \tag{C-Const}$$

$$\frac{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e_1 \Downarrow \langle \Lambda_1, g_1 \rangle : \delta \qquad \Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e_2 \Downarrow \langle \Lambda_2, g_2 \rangle : \delta}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e_1 + e_2 \Downarrow \langle \Lambda_1 \cdot \Lambda_2,\ g_1 + g_2 \rangle : \delta} \tag{C-Add}$$

$$\frac{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e_1 \Downarrow \langle \Lambda_1, g_1 \rangle : \delta \qquad \Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e_2 \Downarrow \langle \Lambda_2, g_2 \rangle : \delta}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e_1 \odot e_2 \Downarrow \langle \Lambda_1 \cdot \Lambda_2,\ g_1 \odot g_2 \rangle : \delta} \tag{C-Had}$$

$$\frac{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e \Downarrow \langle \Lambda, g \rangle : \delta}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash \phi(e) \Downarrow \langle \Lambda, \phi(g) \rangle : \delta} \tag{C-Elem}$$

$$\frac{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e \Downarrow \langle \Lambda, g \rangle : \delta \qquad |\delta| = |\delta'|}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash \mathsf{reshape}(e, \delta') \Downarrow \langle \Lambda, \mathsf{reshape}(g, \delta') \rangle : \delta'} \tag{C-Reshape}$$

$$\frac{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e \Downarrow \langle \Lambda, g \rangle : (d_1, \ldots, d_k) \qquad \pi \in S_k}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash \mathsf{transpose}(e, \pi) \Downarrow \langle \Lambda, \mathsf{transpose}(g, \pi) \rangle : (d_{\pi(1)}, \ldots, d_{\pi(k)})} \tag{C-Transpose}$$

$$\frac{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e \Downarrow \langle \Lambda, g \rangle : \delta}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash \mathsf{norm}(e) \Downarrow \langle \Lambda, \mathsf{norm}(g) \rangle : \delta} \tag{C-Norm}$$

$$\frac{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e \Downarrow \langle \Lambda, g \rangle : (\bar{\delta}, n) \qquad \Pi(T) = (m, n) \qquad T \in \mathcal{T}_{\mathrm{res}}}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash \mathsf{linear}(T, e) \Downarrow \langle \Lambda, \mathsf{linear}_{\mathrm{res}}(T, g) \rangle : (\bar{\delta}, m)} \tag{C-LinRes}$$

$$\frac{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e \Downarrow \langle \Lambda, g \rangle : (\bar{\delta}, n) \qquad \Pi(T) = (m, n) \qquad T \in \mathcal{T}_{\mathrm{str}}}{\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash \mathsf{linear}(T, e) \Downarrow \langle \Lambda \cdot K_\chi(T), \mathsf{linear}_{\mathrm{str}}(T, g) \rangle : (\bar{\delta}, m)} \tag{C-LinStr}$$

Step-body compilation. For a well-typed step body

$$F = \big(e^{out}, \{e_q^{state}\}_{q \in \mathsf{StateId}}\big),$$

suppose

$$\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e^{out} \Downarrow \langle \Lambda_0, g^{out} \rangle : \delta^{out}$$

and for the ordered state identifiers $q_1 \prec \cdots \prec q_m$,

$$\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e_{q_i}^{state} \Downarrow \langle \Lambda_i, g_i \rangle : \Sigma(q_i) \qquad (1 \le i \le m).$$

Then

$$\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash F \Downarrow \big\langle \Lambda_0 \cdot \Lambda_1 \cdots \Lambda_m,\ \big(g^{out}, \langle (q_1, g_1), \ldots, (q_m, g_m) \rangle\big) \big\rangle. \tag{C-Step}$$

Intuition. All non-linear operators translate homomorphically and do not contribute keys. Only a streamed linear operator appends keys to the global stream. The order of keys in Λ is therefore exactly the left-to-right evaluation order of streamed parameter use across the whole step body.

2.8 Target Runtime Semantics

The target runtime semantics executes a target expression against the resident store R, the KV store D, and a *remaining key stream* Σ. Rather than carrying a numeric stream pointer, the semantics threads the unconsumed suffix of the key stream. This makes exact key consumption explicit.

Expression evaluation. The judgment

$$\langle g, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v, \Sigma'\rangle$$

means that target expression g, executed with current input x, incoming state s, and remaining stream Σ, produces value v and leaves remaining stream Σ'.

The base and non-linear cases are standard:

$$\langle X, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle x, \Sigma\rangle, \qquad \langle \mathsf{State}(q), x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle s(q), \Sigma\rangle, \qquad \langle \alpha, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle \alpha, \Sigma\rangle.$$

Binary operators evaluate left-to-right:

$$\frac{\langle g_1, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v_1, \Sigma_1\rangle \qquad \langle g_2, x, s, \Sigma_1\rangle \Downarrow^{\chi}_{R,D} \langle v_2, \Sigma_2\rangle}{\langle g_1 + g_2, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v_1 + v_2, \Sigma_2\rangle} \quad \text{(Q-Add)}$$

$$\frac{\langle g_1, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v_1, \Sigma_1\rangle \qquad \langle g_2, x, s, \Sigma_1\rangle \Downarrow^{\chi}_{R,D} \langle v_2, \Sigma_2\rangle}{\langle g_1 \odot g_2, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v_1 \odot v_2, \Sigma_2\rangle} \quad \text{(Q-Had)}$$

$$\frac{\langle g, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v, \Sigma'\rangle}{\langle \phi(g), x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle \phi(v), \Sigma'\rangle} \quad \text{(Q-Elem)}$$

$$\frac{\langle g, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v, \Sigma'\rangle}{\langle \mathsf{reshape}(g, \delta), x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle \mathsf{reshape}(v, \delta), \Sigma'\rangle} \quad \text{(Q-Reshape)}$$

$$\frac{\langle g, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v, \Sigma'\rangle}{\langle \mathsf{transpose}(g, \pi), x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle \mathsf{transpose}(v, \pi), \Sigma'\rangle} \quad \text{(Q-Transpose)}$$

$$\frac{\langle g, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v, \Sigma'\rangle}{\langle \mathsf{norm}(g), x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle \mathsf{norm}(v), \Sigma'\rangle} \quad \text{(Q-Norm)}$$

Resident-tensor linear operators do not consume stream entries:

$$\frac{\langle g, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v, \Sigma'\rangle \qquad T \in \mathcal{T}_{\mathrm{res}}}{\langle \mathsf{linear}_{\mathrm{res}}(T, g), x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \big\langle \mathsf{Linear}\big(R(T), v\big), \Sigma'\big\rangle} \quad \text{(Q-LinRes)}$$

Streamed-tensor linear operators first evaluate their operand, then consume exactly the next key fragment for that tensor:

$$\frac{\langle g, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle v, K_{\chi}(T) \cdot \Sigma'\rangle \qquad \mathsf{Split}_{\chi,T}(v) = \langle v_1, \ldots, v_{r_T}\rangle \qquad C_i = D[\mathsf{key}_{\chi}(T, i)] \text{ for } 1 \le i \le r_T}{\langle \mathsf{linear}_{\mathrm{str}}(T, g), x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \big\langle \sum_{i=1}^{r_T} \mathsf{Linear}(C_i, v_i), \Sigma'\big\rangle} \quad \text{(Q-LinStr)}$$

Step-plan evaluation. For a target step plan

$$Q = \big(g^{out}, \langle (q_1, g_1), \ldots, (q_m, g_m)\rangle\big),$$

define

$$\mathsf{Eval}_{\mathrm{tgt}}(Q, R, D, x, s, \Sigma) = ((y, s'), \Sigma')$$

if

$$\langle g^{out}, x, s, \Sigma\rangle \Downarrow^{\chi}_{R,D} \langle y, \Sigma_0\rangle$$

and for each $1 \le i \le m$,

$$\langle g_i, x, s, \Sigma_{i-1}\rangle \Downarrow^{\chi}_{R,D} \langle v_i, \Sigma_i\rangle,$$

with $\Sigma_{-1} = \Sigma_0$ only for notational convenience and $\Sigma' = \Sigma_m$, and where

$$s'(q_i) = v_i \quad \text{for } 1 \le i \le m.$$

All state expressions are evaluated against the same incoming state s. This matches the source semantics of simultaneous next-state definition.

2.9 Basic Properties

Lemma 2.1 (Linear decomposition). *Let $\Theta(T) \in \mathcal{T}_{(m,n)}$ and let*

$$\Theta(T) = \mathsf{Concat}(C_1, \ldots, C_r)$$

be any column-wise decomposition. For any

$$v \in \mathcal{T}_{(\delta,n)}$$

let

$$\mathsf{Split}(v) = \langle v_1, \ldots, v_r\rangle$$

be the matching partition of the last dimension. Then

$$\mathsf{Linear}\big(\Theta(T), v\big) = \sum_{i=1}^{r} \mathsf{Linear}(C_i, v_i).$$

Proof. The dense linear operator sums contributions along the last dimension. Partitioning that summation into the contiguous column blocks corresponding to $C_1, \ldots, C_r$ yields exactly the right-hand side. □

Lemma 2.2 (Chunk semantics equals dense semantics). *Let Θ be a dense parameter map and let M be any chunk realization of Θ under χ. Then for every well-typed source expression $\Gamma \vdash e : \delta$,*

$$[\![e]\!]_{x,s,\Theta} = [\![e]\!]^{\chi}_{x,s,M}.$$

Hence for every well-typed step body F,

$$\mathsf{Eval}_{\mathrm{src}}(F, \Theta, x, s) = \mathsf{Eval}_{\chi}(F, M, x, s).$$

Proof. By structural induction on e.

All base cases and all non-linear operators are immediate because the dense and chunk semantics use identical clauses.

For the linear case, write

$$\Theta(T) = \mathsf{Concat}\big(M(T)_1, \ldots, M(T)_{r_T}\big).$$

By the induction hypothesis on the operand,

$$[\![e]\!]_{x,s,\Theta} = [\![e]\!]^{\chi}_{x,s,M}.$$

Applying Lemma 2.1 to this common operand value gives

$$\mathsf{Linear}\big(\Theta(T), [\![e]\!]_{x,s,\Theta}\big) = \sum_{i=1}^{r_T} \mathsf{Linear}\big(M(T)_i, v_i\big),$$

where

$$\mathsf{Split}_{\chi,T}\big([\![e]\!]^{\chi}_{x,s,M}\big) = \langle v_1, \ldots, v_{r_T}\rangle.$$

This is exactly the chunk-semantics clause for $\mathsf{linear}(T, e)$. The step-body statement follows componentwise. □

Lemma 2.3 (Compilation preserves typing). *If*

$$\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash e \Downarrow \langle \Lambda, g\rangle : \delta,$$

then

$$\Gamma \vdash_t g : \delta.$$

If

$$\Gamma; \chi; \mathcal{T}_{\mathrm{res}} \vdash F \Downarrow \langle \Lambda, Q\rangle,$$

then every component of Q is well typed with the same output and state shapes as F.

Proof. Immediate by induction on the compilation derivation. □

2.10 Translator Correctness

Theorem 2.4 (Expression translator correctness). *Assume*

$$\Gamma; \chi; \mathcal{T}_{\text{res}} \vdash e \Downarrow \langle \Lambda, g \rangle : \delta.$$

Let Θ be a dense parameter map, let M be a chunk realization of Θ under χ, and let (R, D) be consistent with $(\Theta, M, \chi, \mathcal{T}_{\text{res}})$. Then for every stream suffix Σ,

$$\langle g, x, s, \Lambda \cdot \Sigma \rangle \Downarrow^{\chi}_{R,D} \left\langle [\![e]\!]^{\chi}_{x,s,M}, \Sigma \right\rangle.$$

In words, the translated target expression consumes exactly its own translated key fragment Λ and returns precisely the chunk-semantics value of the source expression.

Proof. By induction on the compilation derivation.

Base cases. Rules (C-In), (C-State), and (C-Const) translated to the same syntax and contribute the empty stream. The target semantics leaves the input stream unchanged, so the claim is immediate.

Binary operators. Consider rule (C-Add); the Hadamard case is identical. By compilation,

$$\Lambda = \Lambda_1 \cdot \Lambda_2$$

and the target expression is $g_1 + g_2$. Starting from $\Lambda_1 \cdot \Lambda_2 \cdot \Sigma$, the induction hypothesis for e_1 yields

$$\langle g_1, x, s, \Lambda_1 \cdot \Lambda_2 \cdot \Sigma \rangle \Downarrow \left\langle [\![e_1]\!]^{\chi}_{x,s,M}, \Lambda_2 \cdot \Sigma \right\rangle.$$

Applying the induction hypothesis for e_2 to the remaining suffix gives

$$\langle g_2, x, s, \Lambda_2 \cdot \Sigma \rangle \Downarrow \left\langle [\![e_2]\!]^{\chi}_{x,s,M}, \Sigma \right\rangle.$$

Rule (Q-Add) then returns

$$[\![e_1]\!]^{\chi}_{x,s,M} + [\![e_2]\!]^{\chi}_{x,s,M} = [\![e_1 + e_2]\!]^{\chi}_{x,s,M},$$

with final suffix Σ.

Unary non-linear operators. Rules (C-Elem), (C-Reshape), (C-Transpose), and (C-Norm) follow directly from the induction hypothesis and the corresponding target semantic rule.

Resident linear operators. Suppose the last compilation rule is (C-LinRes). By induction on the operand,

$$\langle g, x, s, \Lambda \cdot \Sigma \rangle \Downarrow \left\langle [\![e]\!]^{\chi}_{x,s,M}, \Sigma \right\rangle.$$

Since (R, D) is consistent and $T \in \mathcal{T}_{\text{res}}$,

$$R(T) = \Theta(T) = M(T)_1.$$

Applying rule (Q-LinRes) yields

$$\mathsf{Linear}\big(R(T), [\![e]\!]^{\chi}_{x,s,M}\big) = \mathsf{Linear}\big(M(T)_1, [\![e]\!]^{\chi}_{x,s,M}\big) = [\![\mathsf{linear}(T, e)]\!]^{\chi}_{x,s,M},$$

and the stream suffix remains Σ.

Streamed linear operators. Suppose the last compilation rule is (C-LinStr). Then

$$\Lambda = \Lambda_0 \cdot K_{\chi}(T),$$

where Λ_0 is the operand stream fragment. By the induction hypothesis on the operand,

$$\langle g, x, s, \Lambda_0 \cdot K_{\chi}(T) \cdot \Sigma \rangle \Downarrow \left\langle [\![e]\!]^{\chi}_{x,s,M}, K_{\chi}(T) \cdot \Sigma \right\rangle.$$

Because (R, D) is consistent, for each $1 \le i \le r_T$,

$$D[\mathsf{key}_{\chi}(T, i)] = M(T)_i.$$

Let

$$\mathsf{Split}_{\chi,T}\big([\![e]\!]^{\chi}_{x,s,M}\big) = \langle v_1, \ldots, v_{r_T} \rangle.$$

Applying rule (Q-LinStr) gives

$$\sum_{i=1}^{r_T} \mathsf{Linear}\big(M(T)_i, v_i\big) = [\![\mathsf{linear}(T, e)]\!]^{\chi}_{x,s,M},$$

and the remaining suffix is exactly Σ.

This completes the induction. □

Theorem 2.5 (Step-plan correctness). *Assume*

$$\Gamma; \chi; \mathcal{T}_{\text{res}} \vdash F \Downarrow \langle \Lambda, Q \rangle,$$

where F is well typed. Let Θ be a dense parameter map, let M be a chunk realization of Θ under χ, and let (R, D) be consistent with $(\Theta, M, \chi, \mathcal{T}_{\text{res}})$. Then for every stream suffix Σ,

$$\mathsf{Eval}_{\text{tgt}}(Q, R, D, x, s, \Lambda \cdot \Sigma) = \big(\mathsf{Eval}_{\chi}(F, M, x, s), \Sigma\big).$$

In particular, for closed execution with no extra suffix,

$$\mathsf{Eval}_{\text{tgt}}(Q, R, D, x, s, \Lambda) = \big(\mathsf{Eval}_{\chi}(F, M, x, s), \epsilon\big).$$

Proof. Let

$$Q = \big(g^{out}, \langle (q_1, g_1), \ldots, (q_m, g_m) \rangle\big)$$

and let

$$\Lambda = \Lambda_0 \cdot \Lambda_1 \cdots \Lambda_m$$

be the concatenation produced by rule (C-Step), where

$$\Gamma; \chi; \mathcal{T}_{\text{res}} \vdash e^{out} \Downarrow \langle \Lambda_0, g^{out} \rangle$$

and

$$\Gamma; \chi; \mathcal{T}_{\text{res}} \vdash e^{state}_{q_i} \Downarrow \langle \Lambda_i, g_i \rangle.$$

Apply Theorem 2.4 first to e^{out}, then successively to $e^{state}_{q_1}, \ldots, e^{state}_{q_m}$. Each application consumes exactly the next fragment of the stream and leaves the remainder for the next expression. The value returned for each component is exactly its chunk-semantics value. By definition of $\mathsf{Eval}_{\chi}(F, M, x, s)$ and $\mathsf{Eval}_{\text{tgt}}$, the resulting output and next-state map coincide, and the final remaining suffix is Σ. □

Corollary 2.6 (End-to-end semantic preservation). *Under the assumptions of Theorem 2.5,*

$$\mathsf{Eval}_{\text{tgt}}(Q, R, D, x, s, \Lambda) = \big(\mathsf{Eval}_{\text{src}}(F, \Theta, x, s), \epsilon\big).$$

Equivalently, the translated chunk-streaming plan produces exactly the same output tensor and next-state tensors as the original dense tensor-level step body, while consuming exactly the key stream predicted by compilation.

Proof. Combine Theorem 2.5 with Lemma 2.2. □

3 EXTRACTING REPEATED STEP BODIES FROM COMPUTATIONAL GRAPHS

This appendix describes one practical procedure for extracting the repeated step body

$$F = \big(e^{out}, \{e^{state}_q\}_{q \in \mathsf{StateId}}\big)$$

from an exported computational graph. The goal of the procedure is not to recover every possible loop form in arbitrary graphs, but to recover the repeated-step structure that appears in the workloads targeted by KVTensor, such as autoregressive decoding and diffusion denoising.

Setting. We assume the model has been exported to a standard computational-graph format, such as ONNX, and represented as a directed graph

$$G = (V, E).$$

Each node $v \in V$ denotes one operator instance and carries metadata including

- operator type,
- input and output shapes,
- referenced parameter tensors, if any,
- node name and parameter names from the exported graph.

The raw exported graph is often acyclic. Repeated structure appears as many indexed copies of the same operator pattern, for example one copy per transformer layer or one copy per denoising stage. Our task is to fold these copies into one repeated template and then extract the step body from that template.

3.1 Overview

The extraction procedure has four stages.

(1) Export the model to a computational graph G.
(2) Normalize repeated operator instances by removing concrete layer or step indices from names, so that repeated copies become comparable.
(3) Fold equivalent nodes into a quotient graph G^*, which turns repeated unrolled structure into a compact cyclic template.
(4) Identify a candidate repeated region in G^* by strongly connected component analysis, and recover from it the repeated step body F.

The key idea is that exported graphs often encode repetition by duplication rather than by explicit loops. After normalization and folding, the repeated pattern becomes visible as a cycle in the quotient graph.

3.2 Stage 1: Exported Graph

We first obtain a computational graph from a front-end compiler such as ONNX. At this stage, repeated structure is usually still unrolled. For example, in a transformer with L layers, the graph may contain many operator instances that differ only in parameter names such as

$$W_Q^{(0)},\ W_Q^{(1)},\ \ldots,\ W_Q^{(L-1)},$$

or node names such as

`layer_0/attn/qkv`, `layer_1/attn/qkv`, ...

but have the same operator type and compatible shape signature.

3.3 Stage 2: Normalization of Repeated Instances

To expose repetition, we normalize operator instances before folding. For each node $v \in V$, we define a normalized label

$$\mathsf{Norm}(v).$$

This label retains the information needed to preserve computation, but removes concrete indices that merely distinguish repeated copies. In our implementation, the normalized label contains:

- operator type,
- input and output shape signature,
- normalized parameter names, obtained by stripping layer or step indices,
- a coarse dependency signature derived from predecessor operator types.

For example, parameter names

`layers.0.mlp.w1`, `layers.1.mlp.w1`

are both normalized to a common template name such as

`layers.*.mlp.w1`.

Likewise, repeated operator names differing only in layer index are mapped to one template label.

This normalization does not merge arbitrary nodes. Two nodes are considered candidates for folding only if they have the same normalized operator type, compatible shape signature, and compatible local dependency pattern.

3.4 Stage 3: Folding into a Quotient Graph

After normalization, we construct a quotient graph

$$G^* = (V^*, E^*)$$

by merging nodes that share the same normalized label and compatibility conditions above. Each node in V^* therefore represents an equivalence class of repeated operator instances from the original graph.

Edges in the quotient graph are induced as follows. If there is an edge in the original graph from some node in equivalence class A to some node in equivalence class B, then G^* contains an edge

$$A \to B.$$

The effect of this folding is that repeated unrolled structure collapses into a compact template. What was originally a long chain of repeated copies can now become cyclic. Intuitively, the quotient graph expresses the operator pattern of one repeated step rather than one fully unrolled execution.

Why a cycle appears. In repeated-step workloads, the same structural pattern is reused across iterations. In an unrolled graph this appears as many copies arranged in sequence. After folding those copies into one template node per repeated operator kind, edges from later copies back to the same normalized operator classes induce cycles in G^*. These cycles are the graph-theoretic signature of repeated execution.

3.5 Stage 4: Cycle Detection and Step Extraction

Once the quotient graph G^* is built, we identify its strongly connected components (SCCs). We use Tarjan's algorithm, which runs in linear time in the size of the graph,

$$O(|V^*| + |E^*|).$$

An SCC with more than one node, or a self-loop, indicates a cyclic repeated region.

Let

$$C_1, \ldots, C_k$$

be the SCCs of G^*. We select a candidate SCC corresponding to the repeated inference step using the following criteria:

- it is cyclic,
- it dominates the main arithmetic path of the model,

Algorithm 1 Extract repeated step body from an exported computational graph

Require: Exported computational graph $G = (V, E)$
Ensure: Repeated step body $F = \left(e^{out}, \{e_q^{state}\}_{q \in \mathsf{StateId}}\right)$

```
for each node v ∈ V do
    compute normalized label Norm(v)
end for
construct quotient graph G* by merging nodes with compatible normalized labels
compute strongly connected components of G* using Tarjan's algorithm
select a candidate cyclic component C corresponding to the repeated inference region
identify incoming edges of C as step inputs
identify outgoing edges of C as step outputs
identify recurring boundary values as state slots StateId
reconstruct a step-level operator DAG inside C and derive
    F = (e^out, {e_q^state}_{q ∈ StateId})
return F
```

- it contains the repeated parameterized operators, for example attention or MLP blocks,
- its incoming and outgoing edges are consistent with one-step state transition.

In practice, this candidate SCC usually corresponds to the repeated layer stack or repeated denoising block.

Recovering inputs, outputs, and state. After selecting the candidate repeated SCC C, we classify its boundary edges.

- Incoming edges from outside C define the current-step inputs.
- Outgoing edges leaving C define the emitted output of the step.
- Boundary values that are both consumed by C and reproduced by C across iterations are treated as persistent state and become the members of StateId.

This produces the repeated step body

$$F = \left(e^{out}, \{e_q^{state}\}_{q \in \mathsf{StateId}}\right),$$

where e^{out} is the output expression of one step and each e_q^{state} is the update expression for one persistent state slot.

3.6 Practical Notes

Autoregressive decoding. For autoregressive models, the repeated step usually corresponds to one token-generation iteration. The persistent state consists of the runtime state carried across tokens, such as cache-like state or other recurrent buffers, while the read-only model weights remain parameters.

Diffusion. For diffusion models, the repeated step corresponds to one denoising iteration. The persistent state may include the evolving latent or other iteration-carried tensors.

Heuristic nature. The extraction procedure is intentionally practical rather than universal. It works well for models whose repeated structure is exported as indexed copies of one template. More irregular graphs may require stronger normalization rules or user guidance.

Relation to the main formal system. The formal translation in Section 3 starts from the repeated step body F. The extraction procedure described here is only a front-end that recovers F from an exported graph. It does not affect the semantics-preserving translation itself.